```
$ php getopt1.php -o
array(1) {
  ["o"]=>
  bool(false)
}
```

It might seem a bit strange that `getopt()` confirms `-o` was supplied by assigning a `false` value to the `"o"` array key, but this just means that `-o` exists without any additional data. *nix programs often have other input associated with their switches, and in order to capture that extra data, we need to change the way we call `getopt()`:

```
// getopt2.php
var_dump(getopt('o:'));
```

Notice the colon `:` after the `o`. This makes `-o` capture any text that immediately follows it (after zero or more space/tab characters):

```
$ php getopt2.php -o'some data'
array(1) {
  ["o"]=>
  string(9) "some data"
}
```

To specify long-form options, we add a second argument to `getopt()`, but this time it is an array of option names but still uses a colon for capturing values after the option. While we're at it, we'll add a second short option and make the capturing of `-o` optional (by specifying with a double colon `::`) like this:

```
// getopt3.php
var_dump(getopt('xo::', ['bacon:']));
$ php getopt3.php -x -o=foo  --bacon 'is tasty'
array(3) {
  ["x"]=>
  bool(false)
  ["o"]=>
  string(3) "foo"
  ["bacon"]=>
  string(8) "is tasty"
}
```